

MIAE 215 · Comprehensive Topic Guide (Part 4)

Control Statements, Logic Flow, Flowcharts & Robotics Decision Systems

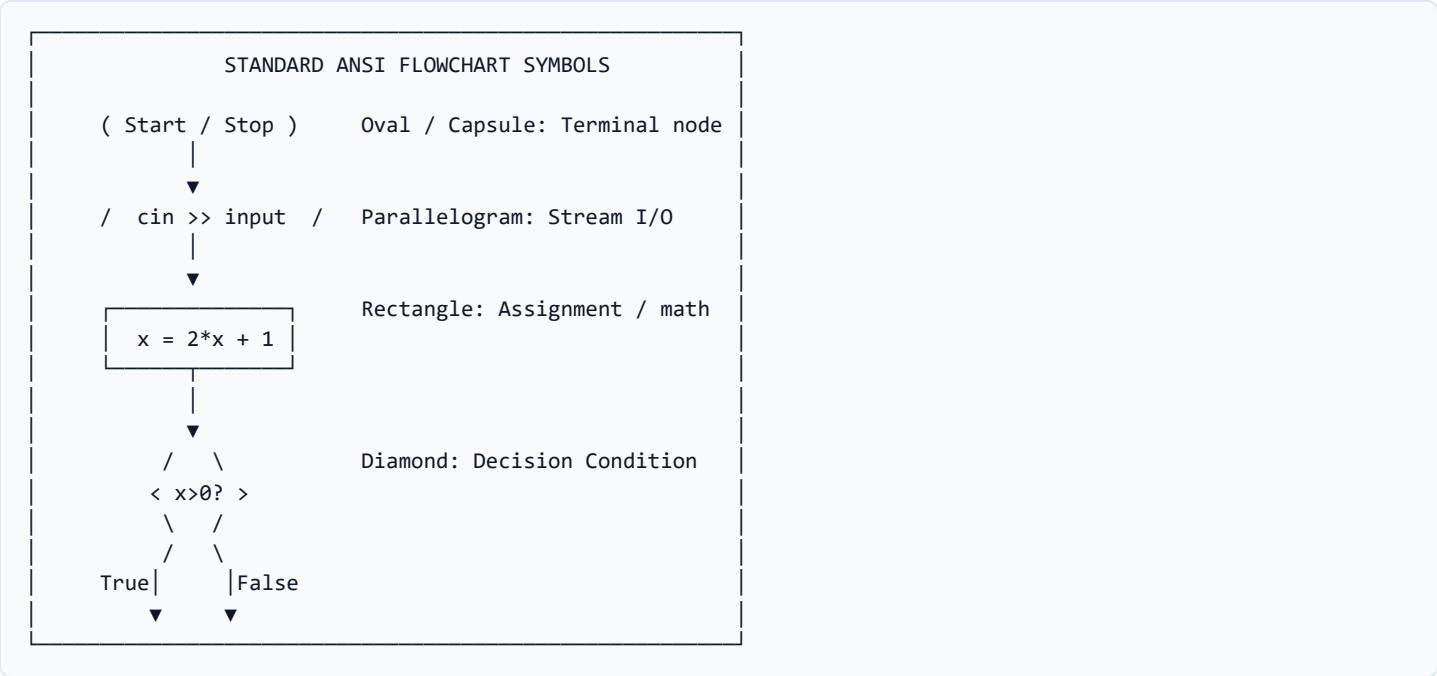
Concordia University · Department of Mechanical, Industrial & Aerospace Engineering (MIAE)

Table of Contents

- 1. [Algorithmic Flowcharts & Visual Logic Modeling](#)
- 2. [Selection Control Structures: Single `if`, Dual `if-else`, and Multi-Way Cascades](#)
- 3. [Relational Comparison Operators & Dynamic State Evaluation](#)
- 4. [The Assignment vs. Equality Cardinal Trap](#)
- 5. [Floating-Point Decision Branching: The Epsilon \(\$\epsilon\$ \) Tolerance Architecture](#)
- 6. [Boolean Logic Gates: AND \(`&&` \), OR \(`||` \), and Inversion \(`!` \)](#)
- 7. [Short-Circuit Evaluation & Zero-Division Safety Guards](#)
- 8. [Nested Branches, The Dangling `else` Ambiguity & Scope Braces](#)
- 9. [Robotics Sensor & Actuator Decision Systems](#)
- 10. [Teacher Lesson Code Walkthrough: `control\_statements1.cpp` & Assignment 1 Problems](#)
- 11. [Week 3 In-Person Lecture Deep-Dive: Input Stream Processing, Negative Exit Guards & Double Accumulation](#)

1. Algorithmic Flowcharts & Visual Logic Modeling

Before writing a single line of C++ code, engineers design and verify algorithmic logic using **flowcharts**. A flowchart provides a formal graphic blueprint mapping decision forks, hardware states, and execution pathways.



Flowgorithm in MIAE 215

Prof. Gordon provides **Flowgorithm** in [05 - Software & Flowcharts/](#) :

- `flow_chart1_simple_program.fprg` : Linear sequence execution, variable initialization, and output display.
- `flow_chart2_if_else.fprg` : Conditional decision forks representing robotic obstacle detection and threshold branching.

2. Selection Control Structures: Single `if` , Dual `if-else` , and Multi-Way Cascades

Selection statements allow a computer program to select between alternate code paths based on Boolean truth evaluations.

1. Single Selection: `if`

Executes an enclosed block of statements if and only if the test condition evaluates to `true` (any non-zero value):

```
if (temperature > 100.0) {
    cout << "Warning: Reactor coolant exceeds boiling threshold!\n";
}
```

2. Dual Selection: `if-else` (The "Fork in the Road")

Provides an alternative default execution path when the condition evaluates to `false` (zero):

```
if (distance_to_wall > 0.5) {
    motor_forward();
} else {
    motor_stop(); // Default safe action
}
```

3. Multi-Way Selection: Cascading `else if`

Evaluates a sequence of mutually exclusive conditions from top to bottom. Execution halts at the first matching branch:

```
if (score >= 90.0) {
    grade = 'A';
} else if (score >= 80.0) {
    grade = 'B';
} else if (score >= 70.0) {
    grade = 'C';
} else {
    grade = 'F';
}
```

3. Relational Comparison Operators & Dynamic State Evaluation

Relational operators compare two operands and produce a Boolean result: `true` (1) or `false` (0).

Operator	Mathematical Meaning	True Example	False Example
>	Greater Than	$5 > 3 \implies 1$	$3 > 5 \implies 0$
<	Less Than	$2 < 8 \implies 1$	$8 < 2 \implies 0$
>=	Greater Than or Equal To	$4 >= 4 \implies 1$	$3 >= 4 \implies 0$
<=	Less Than or Equal To	$5 <= 7 \implies 1$	$9 <= 7 \implies 0$
==	Exactly Equal To	$10 == 10 \implies 1$	$10 == 11 \implies 0$
!=	Not Equal To	$7 != 9 \implies 1$	$7 != 7 \implies 0$

Dynamic Variable Re-Evaluation (Teacher Demonstration)

In `control_statements1.cpp`, Prof. Gordon emphasizes that condition evaluations depend on the **exact line of execution** in RAM:

```
int i = 1, k = 2;

if (k > i) {
    cout << "\nk is greater than i"; // EXECUTES (2 > 1 is true)
}

i = 3; // Variable state in RAM changes!

if (k > i) {
    cout << "\nk is greater than i"; // DOES NOT EXECUTE (2 > 3 is false)
}
```

4. The Assignment vs. Equality Cardinal Trap

By far the most catastrophic, silent bug in C++ programming is confusing the **assignment operator** (`=`) with the **relational equality operator** (`==`).

INTENDED COMPARISON:

```
if (x == 5)
```

```
Reads x from RAM
Tests if x equals 5
Evaluates to TRUE/FALSE
```

CATASTROPHIC ACCIDENTAL ASSIGNMENT:

```
if (x = 5)
```

```
Overwrites x in RAM to 5
Evaluates condition to 5
5 is non-zero → TRUE!
```

```
int pressure = 0; // Safe pressure

if (pressure = 100) { // BUG: Assigns 100 to pressure! 100 is non-zero (TRUE)!
    emergency_shutdown(); // ALWAYS EXECUTES, even when pressure was 0!
}
```

Teacher Exam Warning / Pitfall:

The statement `if (x = 5)` is **syntactically legal C++**! The compiler will NOT produce an error. It overwrites `x` with `5`, and because `5` is non-zero, the `if` block executes every single time. Always double-check for `==` in decision conditions!

5. Floating-Point Decision Branching: The Epsilon (ϵ) Tolerance Architecture

As demonstrated in Part 1 and Part 3, binary floating-point representation cannot represent decimal fractions exactly.

The Round-Off Trap in `if` Statements

In `control_statements1.cpp`, Prof. Gordon presents:

```
double x = 1.0e-30;

// NAIVE CONDITION:
if (x == 0.0) {
    cout << "x is zero"; // WILL NEVER EXECUTE!
}
```

Although 10^{-30} is physically zero for all engineering measurements, `x == 0.0` requires every single one of the 64 bits to be zero.

The Epsilon Tolerance Approach

Define a small tolerance constant ϵ (e.g., 10^{-9}):

```
double eps = 1.0e-9;

// Checking if x is approximately zero:
if (abs(x) < eps) {
    cout << "x is zero within engineering tolerance\n";
}

// Checking if x is approximately equal to a target (e.g., 5.5):
if (abs(x - 5.5) < eps) {
    cout << "x is approximately equal to 5.5\n";
}

// Checking if x is approximately less than or equal to 5.5:
if ((x < 5.5) || (abs(x - 5.5) < eps)) {
    cout << "x is approximately <= 5.5\n";
}
```

6. Boolean Logic Gates: AND (`&&`), OR (`||`), and Inversion (`!`)

To evaluate complex engineering states (e.g., "altitude > 1000 AND velocity < 250 "), simple relational comparisons are joined using Boolean logical operators.

BOOLEAN LOGICAL GATES

```
Logical AND (&&) : True ONLY IF BOTH operands true
Logical OR (||) : True IF AT LEAST ONE operand true
Logical NOT (!) : Inverts Boolean truth value
```

Comprehensive Truth Table

Condition <i>A</i>	Condition <i>B</i>	$A \text{ \&\& } B$	$A \text{ } B$	$\text{!}A$
false (0)	false (0)	false (0)	false (0)	true (1)
false (0)	true (1)	false (0)	true (1)	true (1)
true (1)	false (0)	false (0)	true (1)	false (0)
true (1)	true (1)	true (1)	true (1)	false (0)

```
int i = 1, k = 2;

// Multiple logical conditions chained together:
if ((i < k) && (k <= 3) && (i < 10)) {
    cout << "All three physical conditions are simultaneously satisfied.\n";
}
```

7. Short-Circuit Evaluation & Zero-Division Safety Guards

- C++ evaluates logical expressions **strictly from left to right** and employs **short-circuit evaluation**:
- In `A && B`: If *A* evaluates to `false`, the entire expression cannot possibly be true. *B* **is never evaluated!**
 - In `A || B`: If *A* evaluates to `true`, the entire expression is already guaranteed true. *B* **is never evaluated!**

Guarding Against Division-by-Zero

This hardware guarantee allows engineers to write foolproof safety guards in a single line:

```
double denominator = 0.0;
double numerator = 100.0;

// SHORT-CIRCUIT SAFETY GUARD:
if (denominator != 0.0 && (numerator / denominator > 10.0)) {
    cout << "Threshold exceeded!\n";
}
```

Because `denominator != 0.0` evaluates to `false`, the CPU **halts evaluation immediately**. The division `numerator / denominator` is never executed, preventing a fatal hardware division-by-zero crash!

8. Nested Branches, The Dangling `else` Ambiguity & Scope Braces

When nesting `if` and `if-else` blocks without explicit braces `{ }`, the C++ compiler associates an `else` clause with the **most immediate preceding unclosed `if`**.

```
// DANGEROUS MISLEADING INDENTATION:
if (speed > 0)
    if (speed > 100)
        cout << "Over speeding!";
else
    cout << "Vehicle is stationary?"; // LOGIC ERROR: Binds to (speed > 100), NOT (speed > 0)!
```

The Professional Engineering Rule: Always Use Braces

```
// ROBUST, AMBIGUITY-FREE CODE:
if (speed > 0) {
    if (speed > 100) {
        cout << "Over speeding!";
    }
} else {
    cout << "Vehicle is stationary."; // Clearly binds to (speed > 0)
}
```

9. Robotics Sensor & Actuator Decision Systems

In `control_statements1.cpp`, Prof. Gordon connects control logic to mechanical automation:

```
#include <iostream>
#include <cstdio>
using namespace std;

int main() {
    double robot_distance;

    // Simulate reading from ultrasonic or LIDAR sensor
    cout << "Enter robot distance from obstacle (meters): ";
    cin >> robot_distance;

    if (robot_distance > 0.5) {
        // Send PWM forward voltage to DC motors
        cout << "Path clear. Actuating motors FORWARD.\n";
    } else {
        // Obstacle detected within 0.5m: engage emergency brake
        cout << "Obstacle imminent! Halting motors and engaging BRAKE.\n";
    }

    // Two getchar() calls to handle leftover stream newline
    getchar();
    getchar();

    return 0;
}
```

10. Teacher Lesson Code Walkthrough: `control_statements1.cpp` & Assignment 1 Problems

Assignment 1, Question 5: Threshold Detection with Sentinel Break

Problem: Calculate $f(t) = \sin(t)$ for $t = 0.0, 0.1, 0.2 \dots 10^7$. Terminate the loop immediately when $0.9 < f(t) < 0.93$ using a single `if` statement.

```
#include <iostream>
#include <cstdio>
#include <cmath>

using namespace std;

int main() {
    double f, t;

    for (t = 0.0; t < 1.0e7 + 0.1; t += 0.1) {
        f = sin(t);

        // Single if statement utilizing logical AND (&&)
        if ((f > 0.9) && (f < 0.93)) {
            break; // Immediately exit loop
        }
    }

    cout.precision(6);
    cout << "Loop terminated at t = " << t << " s\n";
    cout << "Function value    f = " << f << "\n";

    return 0;
}
```

Assignment 1, Question 6: Numerical Halving & Oscillatory Convergence

Problem: Input x . Repeatedly divide x by 2 until both $x < 1.0 \times 10^{-7}$ AND $\sin(1/x) > 0.9$. Print final values in scientific notation with 10 digits precision.

```
#include <iostream>
#include <cstdio>
#include <cmath>

using namespace std;

int main() {
    int i;
    double x;

    cout << "Input initial x value: ";
    cin >> x;

    for (i = 0; i < 1000000; i++) {
        // Compound logical break condition
        if ((x < 1.0e-7) && (sin(1.0 / x) > 0.9)) {
            break;
        }
        x = x / 2.0; // Halve search interval
    }

    cout << scientific;
    cout.precision(10);

    cout << "Final x          = " << x << "\n";
    cout << "sin(1.0 / x)      = " << sin(1.0 / x) << "\n";
    cout << "Total Halvings    = " << i << "\n";

    return 0;
}
```

Execution State Trace for Question 6

Iteration (<i>i</i>)	Current <i>x</i>	$x < 1.0 \times 10^{-7}$	$\sin(1/x)$	$\sin(1/x) > 0.9$	Combined Break (<code>&&</code>)	Next <i>x</i>
0	1.000	false	0.841470	false	false	0.500
1	0.500	false	0.909297	true	false (x too large)	0.250
...	...	...	...	...	...	...
24	5.960×10^{-8}	true	-0.342110	false	false	2.980×10^{-8}
Convergence	$x < 10^{-7}$	true	> 0.900000	true	true (Loop Breaks)	Final <i>x</i> output

11. Week 3 In-Person Lecture Deep-Dive: Input Stream Processing, Negative Exit Guards & Double Accumulation

In the **Week 3 In-Person Lecture**, Prof. Gordon introduced interactive stream input processing with loop counters, defensive validation guards, and immediate program abortion strategies.

Lecture Problem Statement

Write a robust C++ engineering program that satisfies two strict operational constraints:

1. **Part (a) Threshold Accumulation:** Prompt the user to input 5 floating-point numbers (`double`) from the keyboard one at a time. Count how many of these doubles have values strictly less than 7.7 (`value < 7.7`). Display the final count upon normal loop completion.
2. **Part (b) Immediate Safety Exit Guard:** If at any point the user inputs a negative number (`value < 0.0`), immediately terminate program execution using either `return 1;` (from `main`) or `exit(1);` (from `<cstdlib>`). No further inputs must be accepted and the summary report must not be printed.

Defensive Exit Strategies: `exit(status)` vs. `return code`

PROGRAM ABORT ARCHITECTURES	
<code>return code;</code>	<code>exit(status_code);</code>
• Defined in language core. • Returns from current func. • Inside <code>main()</code>: exits program and returns code to the OS. • Unwinds local stack objects in C++.	• Defined in <code><cstdlib></code>. • Terminates entire process. • Can be called from any depth (deep inside nested helper functions or sensor drivers). • Flushes buffers, calls <code>atexit</code> handlers, returns code to OS.

In engineering control software, entering abnormal states (e.g., negative pressures, negative Kelvin temperatures, or invalid sensor inputs) often demands an immediate failsafe shutdown.

Complete Production Implementation (w3_11_in_person_example1_count_doubles.cpp)

```
#include <iostream>
#include <stdio>
#include <cmath>
#include <cstdlib> // Required for exit()

using namespace std;

int main() {
    const int TOTAL_INPUTS = 5;
    const double THRESHOLD = 7.7;
    int count_less_than_threshold = 0;
    double current_value = 0.0;

    cout << "=== MIAE 215: Week 3 In-Person Lecture 1 Example ===\n";
    cout << "Enter " << TOTAL_INPUTS << " floating-point numbers one at a time.\n";
    cout << "(Note: Entering any negative value terminates the program immediately)\n\n";

    for (int i = 1; i <= TOTAL_INPUTS; i++) {
        cout << "Enter double #" << i << ": ";
        cin >> current_value;

        // Part (b): Immediate negative input guard
        if (current_value < 0.0) {
            cout << "\n[ALERT] Negative value detected (" << current_value << ")!\n";
            cout << "Stopping program immediately via exit(1)...\n";
            exit(1); // Immediate process termination
        }

        // Part (a): Count values strictly less than 7.7
        if (current_value < THRESHOLD) {
            count_less_than_threshold++;
        }
    }

    cout << "\n===== \n";
    cout << "Total numbers evaluated: " << TOTAL_INPUTS << "\n";
    cout << "Numbers with value < " << THRESHOLD << ": " << count_less_than_threshold << "\n";
    cout << "===== \n";
    cout << "\nDone.\n";

    return 0;
}
```

Execution Trace Analysis

Scenario A: Nominal Execution (All Non-Negative Numbers)

User Inputs: 3.2, 8.4, 7.7, 1.5, 9.0

- $3.2 < 7.7 \implies \text{Count} = 1$
- $8.4 < 7.7 \implies \text{False (Count} = 1)$
- $7.7 < 7.7 \implies \text{False (Strict inequality! Count} = 1)$
- $1.5 < 7.7 \implies \text{Count} = 2$
- $9.0 < 7.7 \implies \text{False (Count} = 2)$
- **Output:** Total numbers evaluated = 5, Numbers < 7.7 = 2. Exit Code = 0.

Scenario B: Premature Termination (Negative Value Entered on Step 3)

User Inputs: 4.5 , 6.1 , -0.05

- Step 1 (4.5): < 7.7 (Count = 1)
- Step 2 (6.1): < 7.7 (Count = 2)
- Step 3 (-0.05): `current_value < 0.0` is `true` .
- **Action:** Alert printed, `exit(1)` executed immediately. Remaining 2 inputs are bypassed. Final summary block is NEVER executed. Exit Code = 1.

Floating-Point Comparison Caveats Revisited

During this lecture, Prof. Gordon reinforced why `current_value == 0.0` or direct equality checks on `double` types are hazardous:

- Due to binary representation limitations (IEEE 754), a calculation producing a theoretical zero may evaluate to `1.0e-17` or `1.0e-30` .
- Direct equality `x == 0.0` evaluates to `false` .
- Always use relational inequalities (`<` , `>` , `<=` , `>=`) or epsilon tolerance bands: `fabs(a - b) < 1.0e-7` .